

ПРАКТИЧЕСКАЯ РАБОТА №1

Объектно-ориентированное программирование на Java

Разработка консольной системы Help Desk

Java 17 • классы и объекты • 4 принципа ООП • коллекции

ФИО студента	
Группа	
Дата выполнения	
Оценка	

Практика ориентирована на понимание архитектуры, а не на механическое копирование кода.

1. Цель и результат работы

Цель: научиться применять базовые средства Java и четыре основных принципа объектно-ориентированного программирования при проектировании небольшой информационной системы.

- создавать классы, объекты, конструкторы и методы;
- защищать состояние объекта с помощью инкапсуляции;
- использовать наследование только там, где существует отношение IS-A;
- применять переопределение методов и полиморфизм;
- создавать абстрактные классы и интерфейсы;
- использовать enum и коллекции List / ArrayList;
- разделять модель, сервисную логику и хранение данных.

Итог: к концу работы у вас будет консольная модель Help Desk, где клиент создаёт заявку, специалист принимает её в работу, меняет статус, закрывает заявку и отправляет уведомления.

2. Исходные условия

Язык	Java 17
Среда	IntelliJ IDEA или другая IDE с поддержкой Java 17
Тип проекта	Console Application
Spring	не используется
Рекомендуемое имя проекта	help-desk-oop

3. Сценарий системы

В Help Desk существуют клиенты, специалисты поддержки и администраторы. Клиент сообщает о проблеме. Система создаёт заявку, после чего заявка проходит жизненный цикл:

Шаг	Статус	Смысл
1	NEW	заявка создана и ожидает обработки
2	IN_PROGRESS	специалист принял заявку в работу
3	RESOLVED	проблема решена
4	CLOSED	заявка закрыта

Инженерная идея: статус нельзя менять произвольно. Сам объект Ticket должен контролировать допустимые переходы.

4. Рекомендуемая структура проекта

```
src/main/java/  
└─ helpdesk/  
    └─ Main.java  
    └─ model/  
        └─ Ticket.java  
        └─ TicketStatus.java  
        └─ User.java  
        └─ Customer.java  
        └─ SupportAgent.java  
        └─ Administrator.java  
    └─ service/  
        └─ NotificationService.java  
        └─ ConsoleNotificationService.java  
        └─ TicketService.java  
    └─ repository/  
        └─ TicketRepository.java
```

Важно: для первой работы разрешено использовать один package, но разделение по пакетам показывает, как проект организуют в реальной разработке.

5. Ход выполнения практической работы

Задание 1. Создание проекта и точки входа

Создайте Java 17 проект и класс Main. Проверьте, что приложение запускается.

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("=== HELP DESK ===");  
    }  
}
```

Ожидаемый вывод:

```
=== HELP DESK ===
```

Задание 2. Первая модель Ticket

Создайте класс Ticket. На первом шаге используйте обычные поля - это позволит увидеть проблему прямого доступа.

```
public class Ticket {  
    long id;  
    String title;  
    String description;  
    String status;  
  
    public Ticket(long id, String title, String description) {  
        this.id = id;  
        this.title = title;  
        this.description = description;  
        this.status = "NEW";  
    }  
}
```

Создайте объект:

```
Ticket ticket = new Ticket(  
    1,  
    "Не работает интернет",  
    "После перезагрузки компьютера пропало подключение"  
);
```

Проверьте: можно ли написать `ticket.status = "БАНАН"`; ? Почему компилятор это допускает и почему для бизнес-системы это опасно?

Задание 3. Инкапсуляция состояния заявки

Сделайте поля закрытыми и предоставьте только необходимые методы чтения.

```
private long id;  
private String title;  
private String description;  
private String status;  
  
public long getId() { return id; }  
public String getTitle() { return title; }  
public String getDescription() { return description; }  
public String getStatus() { return status; }
```

Setter для status не создавайте. Вместо него добавьте методы, выражающие бизнес-действия:

```
public void startProcessing() {  
    status = "IN_PROGRESS";  
}  
  
public void resolve() {  
    status = "RESOLVED";  
}  
  
public void close() {  
    status = "CLOSED";  
}
```

Вопрос: почему `ticket.close()` лучше, чем `ticket.setStatus("CLOSED")`? В ответе используйте понятия "смысл операции" и "контроль состояния".

Задание 4. Контроль допустимых переходов

Запретите нелогичные переходы. Например, нельзя решить заявку, которая ещё не взята в работу.

```
public void resolve() {
    if (!status.equals("IN_PROGRESS")) {
        System.out.println("Ошибка: решить можно только заявку в работе");
        return;
    }
    status = "RESOLVED";
}
```

Самостоятельно добавьте проверки для `startProcessing()` и `close()`.

Задание 5. Enum вместо строк

Создайте перечисление статусов:

```
public enum TicketStatus {
    NEW,
    IN_PROGRESS,
    RESOLVED,
    CLOSED
}
```

Замените `String` status на:

```
private TicketStatus status;
```

Инициализация новой заявки:

```
this.status = TicketStatus.NEW;
```

Пример корректной проверки:

```
if (status != TicketStatus.NEW) {
    System.out.println("В работу можно взять только новую заявку");
    return;
}
status = TicketStatus.IN_PROGRESS;
```

Мини-вывод: enum исключает опечатки типа `"IN_PROGRES"` и ограничивает набор допустимых значений на уровне языка.

Задание 6. Базовый класс User

```
public class User {
    private long id;
    private String name;
    private String email;

    public User(long id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public long getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
}
```

Задание 7. Наследование ролей

Создайте три класса-наследника: Customer, SupportAgent, Administrator.

```
public class Customer extends User {
    public Customer(long id, String name, String email) {
        super(id, name, email);
    }
}
```

По аналогии создайте SupportAgent и Administrator.

Проверка IS-A: Customer является User - да. Ticket является User - нет. Поэтому Ticket не должен наследоваться от User.

Задание 8. Переопределение и полиморфизм

В User добавьте метод performAction(), затем переопределите его в каждом наследнике.

```
public void performAction() {
    System.out.println("Пользователь работает с системой");
}

@Override
public void performAction() {
    System.out.println(getName() + " создаёт заявку");
}
```

В SupportAgent выводите "обрабатывает заявку", в Administrator - "управляет системой".

Создайте список пользователей:

```
List<User> users = new ArrayList<>();
users.add(new Customer(1, "Анна", "anna@mail.ru"));
users.add(new SupportAgent(2, "Сергей", "sergey@helpdesk.ru"));
users.add(new Administrator(3, "Олег", "admin@helpdesk.ru"));

for (User user : users) {
    user.performAction();
}
```

Ключевой вопрос: почему строка user.performAction() одна, а фактически выполняются разные методы? Это главный практический пример полиморфизма.

Задание 9. Абстрактный User

В системе не существует пользователя без конкретной роли. Сделайте User абстрактным:

```
public abstract class User {
    // поля, конструктор и getter-методы
    public abstract void performAction();
}
```

Проверьте, что `new User(...)` теперь создать нельзя. Наследники обязаны реализовать `performAction()`.

Задание 10. Интерфейс уведомлений

Создайте контракт отправки уведомлений:

```
public interface NotificationService {
    void send(String message);
}
```

Реализация для консоли:

```
public class ConsoleNotificationService implements NotificationService {
    @Override
    public void send(String message) {
        System.out.println("[УВЕДОМЛЕНИЕ] " + message);
    }
}
```

Задание 11. TicketService

Создайте сервис, который управляет действиями с заявкой и зависит от интерфейса, а не от конкретного способа уведомления.

```
public class TicketService {
    private final NotificationService notificationService;

    public TicketService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    public void startTicket(Ticket ticket) {
        ticket.startProcessing();
        notificationService.send(
            "Заявка №" + ticket.getId() + " принята в работу"
        );
    }
}
```

Использование:

```
NotificationService notificationService =
    new ConsoleNotificationService();

TicketService ticketService =
    new TicketService(notificationService);

ticketService.startTicket(ticket);
```

Архитектурный смысл: `TicketService` знает, что уведомление можно отправить, но не знает деталей отправки. Позже `ConsoleNotificationService` можно заменить другой реализацией.

Задание 12. Репозиторий заявок

```
public class TicketRepository {
    private final List<Ticket> tickets = new ArrayList<>();

    public void add(Ticket ticket) {
        tickets.add(ticket);
    }

    public List<Ticket> findAll() {
        return new ArrayList<>(tickets);
    }
}
```

Добавьте не менее трёх заявок и выведите их в формате:

```
#1 Не работает интернет | NEW
#2 Ошибка приложения | NEW
#3 Не печатает принтер | NEW
```

Обратите внимание: возврат `new ArrayList<>(tickets)` не позволяет внешнему коду напрямую изменять внутреннюю коллекцию репозитория - это тоже проявление инкапсуляции.

6. Итоговый обязательный сценарий

Соберите созданные классы в единый работающий сценарий:

1. Создать клиента Анна Петрова.
2. Создать специалиста поддержки Сергей Иванов.
3. Создать заявку "Не работает Wi-Fi" с описанием проблемы.
4. Убедиться, что начальный статус - NEW.
5. Принять заявку в работу - IN_PROGRESS.
6. Отметить проблему решённой - RESOLVED.
7. Закрыть заявку - CLOSED.
8. При каждом успешном переходе вывести уведомление.
9. Добавить заявку в TicketRepository.
10. Вывести полный список заявок.

Пример ожидаемого фрагмента консольного вывода:

```
=== HELP DESK ===
Клиент: Анна Петрова
Заявка #1: Не работает Wi-Fi | NEW
[УВЕДОМЛЕНИЕ] Заявка №1 принята в работу
Статус: IN_PROGRESS
[УВЕДОМЛЕНИЕ] По заявке №1 найдено решение
Статус: RESOLVED
[УВЕДОМЛЕНИЕ] Заявка №1 закрыта
Статус: CLOSED
```

7. Зафиксируйте четыре принципа ООП в своём проекте

Принцип	Что найти в проекте	Ваш пример
Инкапсуляция	private-поля Ticket; изменение статуса только через методы	
Наследование	Customer / SupportAgent / Administrator extends User	
Полиморфизм	List<User> и вызов user.performAction()	
Абстракция	abstract User; интерфейс NotificationService	

В отчёте вставьте по одному небольшому фрагменту своего кода для каждого принципа и объясните его в 1-2 предложениях.

8. Самостоятельная работа

Самостоятельная работа продолжает тот же проект. Выполняйте задания без переписывания проекта с нуля. Главное требование - новые возможности должны встраиваться в существующую архитектуру.

Уровень 1. Базовый - укрепляем модель

№	Задание	Что проверяется
1	Добавьте TicketPriority: LOW, MEDIUM, HIGH, CRITICAL. Приоритет задаётся при создании заявки и доступен только для чтения.	enum, инкапсуляция
2	Добавьте статус CANCELLED и метод cancel(). Запретите отмену уже закрытой заявки.	контроль состояния
3	Проверьте title: пустое название заявки создавать нельзя. Выберите: IllegalArgumentException или проверка с сообщением.	валидация
4	Добавьте createdAt типа LocalDateTime и автоматически фиксируйте время создания заявки.	состояние объекта
5	Переопределите toString() у Ticket так, чтобы заявка красиво выводилась одной строкой.	методы Object

Минимум для самостоятельной работы: выполнить любые 4 задания уровня 1.

Уровень 2. Средний - поведение системы

№	Задание	Ограничение
6	SupportAgent должен хранить список назначенных ему заявок и иметь метод assignTicket(Ticket).	при назначении NEW -> IN_PROGRESS
7	В TicketRepository добавьте поиск findById(long id). Если заявки нет - вернуть Optional<Ticket>.	не возвращать null
8	Добавьте метод findByStatus(TicketStatus status), возвращающий список заявок выбранного статуса.	использовать коллекции
9	Добавьте метод findByPriority(TicketPriority priority).	не дублировать лишнюю логику
10	Создайте EmailNotificationService и TelegramNotificationService. TicketService менять при переключении реализации нельзя.	интерфейс + полиморфизм
11	Добавьте метод getStatistics(), выводящий число заявок по каждому статусу.	агрегация данных
12	Запретите Customer выполнять действия специалиста. Не решайте задачу через if по строковому имени роли.	модель ролей

Условие уровня 2: выберите минимум 4 задания, одно из них обязательно №10.

Уровень 3. Повышенный - проектируем как разработчики

№	Задание	Идея
13	Добавьте SLA: для CRITICAL - 30 минут, HIGH - 2 часа, MEDIUM - 8 часов, LOW - 24 часа. Рассчитайте deadline.	бизнес-правило
14	Добавьте историю изменений статуса. Каждая запись хранит	композиция

	старый статус, новый статус и время изменения.	
15	Создайте интерфейс TicketRepository и реализацию InMemoryTicketRepository. TicketService должен зависеть от интерфейса.	зависимость от абстракции
16	Создайте исключения InvalidTicketStateException и TicketNotFoundException и используйте их вместо части println-ошибок.	обработка ошибок
17	Создайте интерфейс PriorityCalculator. Реализуйте DefaultPriorityCalculator и VipPriorityCalculator.	Strategy / полиморфизм
18	Добавьте команду "показать просроченные заявки": текущий момент позже deadline, а заявка ещё не RESOLVED/CLOSED.	работа со временем
19	Добавьте простой AuditService, который записывает в List<String> ключевые действия пользователей.	разделение ответственности
20	Напишите минимум 5 unit-тестов для Ticket: корректный жизненный цикл и запрещённые переходы. Можно использовать JUnit 5.	тестируемость модели

Условие уровня 3: выполнить минимум 3 задания. За качественную реализацию уровня 3 преподаватель может заменить часть устной защиты демонстрацией кода.

9. Творческое самостоятельное задание

Выберите ОДИН вариант и реализуйте его как отдельное расширение Help Desk.

Вариант	Задача
A. IT-служба университета	Типы заявок: сеть, программное обеспечение, оборудование, учётная запись. Добавьте категорию и фильтрацию.
B. Поддержка интернет-магазина	Типы пользователей: покупатель, оператор, старший оператор. Добавьте заявку на возврат/доставку/оплату.
C. Сервисный центр	К заявке привязывается устройство: тип, модель, серийный номер. Продумайте композицию Ticket - Device.
D. Корпоративный Help Desk	Добавьте отдел пользователя и автоматический выбор приоритета на основе категории проблемы.

Требование: в расширении должны явно использоваться минимум три принципа ООП. В отчёте укажите, где именно.

10. Задания при защите

Преподаватель выбирает 1-2 изменения. Студент выполняет их в своём проекте и объясняет результат.

- добавить новый статус REOPENED и определить, из какого состояния в него можно перейти;
- добавить SmsNotificationService без изменения TicketService;
- добавить нового пользователя SeniorSupportAgent;
- запретить закрытие заявки, если она не RESOLVED;
- добавить поиск заявки по части названия;
- объяснить, какой метод выполнится для User user = new SupportAgent(...); user.performAction();
- показать место, где нарушится архитектура, если заменить NotificationService на конкретный ConsoleNotificationService внутри TicketService.

11. Контрольные вопросы

1. Чем класс отличается от объекта?
2. Почему private-поле само по себе ещё не гарантирует хорошую инкапсуляцию?
3. Почему setter для status в Ticket нежелателен?
4. В чём разница между extends и implements?
5. Что означает отношение IS-A? Приведите пример из Help Desk.
6. Что означает отношение HAS-A? Приведите пример, который можно добавить в проект.
7. Зачем используется @Override?
8. Почему List<User> может хранить Customer и SupportAgent?
9. Как Java выбирает конкретную реализацию performAction() во время выполнения?
10. Почему абстрактный класс User нельзя создать через new?
11. Чем интерфейс NotificationService полезнее жёсткой зависимости от ConsoleNotificationService?
12. Почему enum TicketStatus надёжнее строкового статуса?

13. Какой класс должен отвечать за допустимые переходы статуса: Main, TicketService или Ticket? Обоснуйте.
14. Что изменится в архитектуре при переходе с консольных уведомлений на Telegram?
15. Где в созданном проекте можно использовать композицию вместо наследования?

12. Требования к отчёту

11. ФИО и группа.
12. Ссылка на Git-репозиторий (если используется Git).
13. Скриншот структуры проекта.
14. Код Ticket и TicketStatus.
15. Код User и одного наследника.
16. Код NotificationService и одной реализации.
17. Скриншот полного жизненного цикла заявки NEW -> IN_PROGRESS -> RESOLVED -> CLOSED.
18. Заполненная таблица "4 принципа ООП".
19. Перечень выполненных заданий самостоятельной работы.
20. Ответы на контрольные вопросы (кратко, своими словами).

13. Критерии оценивания

Уровень	Требования	Ориентир
Зачёт / 3	Обязательная часть запускается; Ticket, enum, User и наследники; базовая инкапсуляция; жизненный цикл заявки.	60-69%
Хорошо / 4	Всё для 3 + полиморфизм, abstract User, NotificationService, TicketService, Repository, минимум 4 задания самостоятельной работы.	70-84%
Отлично / 5	Всё для 4 + качественное разделение ответственности, минимум 2 реализации уведомлений, задания уровня 2/3, уверенная защита.	85-100%

Оценивается не количество строк кода: важнее, чтобы студент мог объяснить, почему ответственность находится именно в этом классе и какой принцип ООП использован.

14. Чек-лист перед сдачей

- ☐ Проект запускается на Java 17 без ошибок.
- ☐ Поля модели не открыты public без необходимости.
- ☐ Статус Ticket представлен enum.
- ☐ Недопустимые переходы состояния запрещены.
- ☐ User является абстрактным классом.
- ☐ Customer, SupportAgent, Administrator наследуются от User.
- ☐ Полиморфизм показан через коллекцию базового типа или интерфейс.
- ☐ NotificationService является интерфейсом.
- ☐ TicketService не зависит напрямую от конкретного способа уведомлений.
- ☐ Заявки хранятся в коллекции через TicketRepository.
- ☐ Выполнены задания самостоятельной работы.
- ☐ Студент может объяснить четыре принципа ООП на своём коде.

15. Итог работы

В этой работе вы не просто создаёте классы. Вы учитесь распределять ответственность между объектами: Ticket защищает своё состояние; User задаёт общую модель ролей; наследники определяют конкретное поведение; NotificationService скрывает способ доставки уведомлений; TicketService координирует операции; TicketRepository хранит заявки.

Главная мысль: хорошее ООП - это не максимальное количество классов и наследования. Это понятная модель, в которой изменения можно вносить локально, не ломая всю программу.